



DEPARTMENT OF MATHEMATICS

Sleep Health & Lifestyle

A MINI PROJECT REPORT

Submitted as part of the
Self Study / Term Work Component (Semester - II)

Div : D

Submitted by

Sl.No.	Name of the Student	Roll Number	USN
1	N B Varun	451	01FE25BCS272
2	Alfiya Dastageerasab Dhalayat	453	01FE25BCS274
3	Darshankumar Pujari	455	01FE25BCS277
4	Swayam Gaonkar	457	01FE25BCS279
5	Anusha S Wali	459	01FE25BCS281

Under the Guidance of

Manisha Bulbule Ma'am

Faculty Supervisor

Academic Year: 2025 - 2026

01.TABLE OF CONTENTS

Sl.No.	Section	Content Description	Page No.
1	Title Page	Project title, student info	1
2	Table of Contents	Complete index with page references	2
3	Index & Introduction	Dataset description, objectives, variable reference	3
4	Methodology	Statistical techniques and Python tools applied	4
5	Results / Analysis	Key findings, hypothesis testing & correlation	5
6	Experience & Conclusion	Final conclusions and real-world insights	6
7	References	Dataset, Python libraries & academic sources	6
8	Appendix (Python Code)	Full Python notebook · code & visualisations	7

02 Introduction

About the Dataset

The Sleep Health and Lifestyle Dataset is a comprehensive health data collection from 374 participants encompassing 13 variables — covering demographics, sleep metrics, physical health, and lifestyle factors. It was compiled to investigate relationships between daily habits, occupational stress, and sleep quality across a diverse group of working professionals.

Demographic Variables

Person ID · Gender · Age
Occupation · BMI Category

Blood Pressure

Health & Lifestyle Variables

Sleep Duration · Quality of Sleep
Physical Activity · Stress Level

Heart Rate · Daily Steps · Sleep Disorder

Purpose of the Analysis

This project was conducted as part of the 2nd Semester Statistics Mini Project. The primary objective is to understand how lifestyle and occupational factors influence sleep quality among working professionals. Key research questions include: Which variables most strongly predict sleep quality? Are there significant gender differences? What role does stress play? Does sleep disorder status measurably degrade rest quality?

Variable Reference Table

Variable	Type	Description
Quality of Sleep	Numeric (1–10)	Self-reported sleep quality score
Sleep Duration	Numeric (hrs)	Hours of sleep per night
Stress Level	Numeric (1–10)	Self-reported daily stress score
Physical Activity Level	Numeric (min)	Minutes of daily physical activity
Heart Rate	Numeric (bpm)	Resting heart rate in beats per minute
Daily Steps	Numeric	Average number of steps walked per day
Gender	Categorical	Male / Female
Sleep Disorder	Categorical	No Disorder / Insomnia / Sleep Apnea

Report Overview

This report presents a comprehensive statistical analysis of the Sleep Health and Lifestyle dataset, consisting of 374 working professionals recorded across 13 variables — including Gender, Age, Occupation, Sleep Duration, Quality of Sleep, Physical Activity Level, Stress Level, BMI Category, Blood Pressure, Heart Rate, Daily Steps, and Sleep Disorder status. The dataset captures a diverse range of occupations such as Software Engineers, Doctors, Nurses, Teachers, Sales Representatives, Accountants, and more — making it well-suited to study how professional lifestyle influences health and sleep outcomes.

The core objective of this report is to identify which occupations are most affected by stress, poor sleep quality, and sleep disorders, and to understand the key factors driving these outcomes. Structured across 6 sections and 12 tasks, the analysis covers descriptive statistics, data visualisation, probability, hypothesis testing, correlation & regression, and chi-square testing — all implemented using Python

03 Statistical Methods & Tools Applied

This project applies a structured six-stage statistical pipeline. All analysis was performed using Python 3 with Pandas, NumPy, Matplotlib, Seaborn, SciPy, and Scikit-Learn. Each stage targets a specific analytical goal.

01 Descriptive Statistics

Measures of central tendency (mean, median, mode) and dispersion (range, standard deviation, IQR, CV) were computed for all six numeric variables: Sleep Duration, Quality of Sleep, Physical Activity Level, Stress Level, Heart Rate, and Daily Steps. This provides a full snapshot of the dataset's distributions and relative variability.

02 Data Visualisation

Histograms with KDE overlays visualise frequency distributions and detect skewness. Box plots (overall, by Gender, by Sleep disorder) compare medians and identify outliers using the $1.5 \times \text{IQR}$ rule. QQ-Norm plots with Shapiro-Wilk tests assess normality — a prerequisite for choosing the correct parametric test.

03 Probability

Bayes' Theorem quantifies the conditional relationship between Sleep Disorder status and Sleep Quality — defining A = Good Sleep Quality (score ≥ 7) and B = No Sleep Disorder, then computing and verifying prior and posterior probabilities. A Normal Distribution Fit estimates the probability of exceeding a threshold and determines the 90th percentile score.

04 Hypothesis Testing

A one-sample t-test tests whether the sample mean sleep quality significantly differs from a benchmark of 7.0. A two-sample independent t-test (Welch's, after Levene's variance test) compares Male vs Female sleep quality. Both tests evaluate significance at $\alpha = 0.05$ with clear accept/reject decisions stated.

05 Correlation & Regression

Pearson correlation coefficients and p-values are computed between Quality of Sleep and all five numeric predictors. A heatmap visualises the full matrix. Simple Linear Regression then uses Stress Level to predict Quality of Sleep, reporting intercept, slope, R^2 , and a residual plot to validate linearity.

06 Chi-Square Test

The Chi-Square Test for Independence tests two categorical associations: Gender $\times$ Sleep Disorder type, and Gender $\times$ High-Quality Sleep (Quality $\geq$ median). Contingency tables, χ^2 statistics, degrees of freedom, and p-values are reported, with decisions at $\alpha = 0.05$.

04 Key Findings from All Six Sections

1. Descriptive Statistics

Variable	Mean	Median	Std Dev	CV (%)
Sleep Duration	7.132	7.200	0.796	11.2
Quality of Sleep	7.313	7.000	1.197	16.4
Physical Activity Level	59.171	60.000	20.831	35.2
Stress Level	5.385	5.000	1.775	33.0
Heart Rate	70.166	70.000	4.136	5.9
Daily Steps	6816.845	7000.000	1617.916	23.7

● Finding: Physical Activity Level has the highest variability (CV = 35.2%). Heart Rate is the most consistent variable (CV = 5.9%). Quality of Sleep shows a slight right-skew (mean 7.31 > median 7.00, skewness = 0.939).

2. Hypothesis Testing Results

One-Sample t-Test ($H_0: \mu = 7.0$) Sample Mean = 7.313 t-statistic = 5.0544 p-value = 0.0000 Decision: Reject H_0 — The mean sleep quality is significantly higher than the benchmark of 7.0 ($p < 0.05$).	Two-Sample t-Test (Male vs Female) Male Mean = 6.968 Female Mean = 7.665 t = -5.8593 p = 0.0000 Levene p = 0.0006 (Unequal variances) Decision: Reject H_0 — Females have significantly higher sleep quality.
--	--

3. Correlation Analysis

Variable Pair	r	p-value	Significance
Sleep Duration ↔ Quality of Sleep	+0.883	0.0000	Significant
Physical Activity Level ↔ Quality of Sleep	+0.193	0.0002	Significant
Stress Level ↔ Quality of Sleep	-0.899	0.0000	Significant
Heart Rate ↔ Quality of Sleep	-0.660	0.0000	Significant
Daily Steps ↔ Quality of Sleep	+0.017	0.7462	Not Significant

● Finding: Stress Level is the strongest predictor of Quality of Sleep ($r = -0.899$, $p = 0.0000$). Sleep Duration also shows a very strong positive correlation ($r = +0.883$). Daily Steps is the only variable with no significant link to sleep quality ($p = 0.7462$).

Regression: Stress Level → Quality of Sleep Model: Quality of Sleep = 10.577 + (-0.606) × Stress Level $R^2 = 0.808$ — Stress Level explains 80.8% of the variance in sleep quality. Every 1-unit rise in stress reduces sleep quality by 0.606 points.	Chi-Square Test Results Gender × Sleep Disorder: $\chi^2 = 54.3060$, df = 2, p = 0.0000 Decision: Reject H_0 → Attributes are dependent Gender × High Quality Sleep: $\chi^2 = 10.2811$, p = 0.0013 Decision: Reject H_0 → Dependent
---	---

05 Experience

• Experience

Working on the Sleep Health & Lifestyle dataset was a hands-on deep dive into the full data analysis pipeline. Using Python, I implemented twelve structured tasks spanning descriptive statistics, data visualisation (histograms, box plots, Q-Q plots), probability theory, hypothesis testing, correlation, regression, and chi-square testing. Each section involved translating raw data into meaningful statistical outputs and interpreting results beyond just numbers. The project strengthened my ability to choose the right test based on data characteristics — for instance, using Welch's t-test after Levene's test flagged unequal variances, and opting for nonparametric awareness after Shapiro-Wilk confirmed non-normality. Presenting findings in a structured PDF report alongside the code reinforced the importance of clear communication in data science.

Statistical Conclusions

- Stress Level is the single strongest predictor of sleep quality ($r = -0.899$, $p = 0.0000$), accounting for 80.8% of variance in the regression model.
- Female participants scored significantly higher in sleep quality than males (7.665 vs 6.968, $p = 0.0000$). Gender is dependent on both sleep disorder type ($p = 0.0000$) and high-quality sleep status ($p = 0.0013$).
- Sleep Duration and Quality of Sleep are strongly positively correlated ($r = +0.883$) — longer sleep is associated with better rest quality.
- Daily Steps has no significant correlation with sleep quality ($r = 0.017$, $p = 0.7462$) — step count alone does not determine how well a person sleeps.

06 References

- [1] Kaggle. Sleep Health and Lifestyle Dataset.
URL: <https://www.kaggle.com/datasets/uom190346a/sleep-health-and-lifestyle-dataset>
- [2] McKinney, W. (2010). Data Structures for Statistical Computing in Python.
Proceedings of the 9th Python in Science Conference.
- [3] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment.
Computing in Science & Engineering, 9(3), 90-95.
- [4] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12.
- [5] Virtanen, P. et al. (2020). SciPy 1.0. Nature Methods, 17, 261-272.
- [6] Waskom, M. (2021). Seaborn: Statistical Data Visualization. JOSS, 6(60), 3021.

Sleep Health Analysis

Python Notebook — Code & Results

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[2]: df = pd.read_csv('Sleep_health_and_lifestyle_dataset.csv')
```

```
[3]: df
```

```
[3]:
```

	Person ID	Gender	Age	Occupation	Sleep Duration \
0	1	Male	27	Software Engineer	6.1
1	2	Male	28	Doctor	6.2
2	3	Male	28	Doctor	6.2
3	4	Male	28	Sales Representative	5.9
4	5	Male	28	Sales Representative	5.9
..	...	...	...	...	...
369	370	Female	59	Nurse	8.1
370	371	Female	59	Nurse	8.0
371	372	Female	59	Nurse	8.1
372	373	Female	59	Nurse	8.1
373	374	Female	59	Nurse	8.1

	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category \
0	6	42	6	Overweight
1	6	60	8	Normal
2	6	60	8	Normal
3	4	30	8	Obese
4	4	30	8	Obese
..	...	...	...	...
369	9	75	3	Overweight
370	9	75	3	Overweight
371	9	75	3	Overweight
372	9	75	3	Overweight
373	9	75	3	Overweight

	Blood Pressure	Heart Rate	Daily Steps	Sleep Disorder
0	126/83	77	4200	NaN
1	125/80	75	10000	NaN
2	125/80	75	10000	NaN
3	140/90	85	3000	Sleep Apnea
4	140/90	85	3000	Sleep Apnea
..	...	...	...	...
369	140/95	68	7000	Sleep Apnea
370	140/95	68	7000	Sleep Apnea
371	140/95	68	7000	Sleep Apnea
372	140/95	68	7000	Sleep Apnea
373	140/95	68	7000	Sleep Apnea

[374 rows x 13 columns]

```
[4]: df.shape
```

```
[4]: (374, 13)
```

```
[5]: df.head()
```

```
[5]:
```

	Person ID	Gender	Age	Occupation	Sleep Duration	\
0	1	Male	27	Software Engineer	6.1	
1	2	Male	28	Doctor	6.2	
2	3	Male	28	Doctor	6.2	
3	4	Male	28	Sales Representative	5.9	
4	5	Male	28	Sales Representative	5.9	

	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category	\
0	6	42	6	Overweight	
1	6	60	8	Normal	
2	6	60	8	Normal	
3	4	30	8	Obese	
4	4	30	8	Obese	

	Blood Pressure	Heart Rate	Daily Steps	Sleep Disorder
0	126/83	77	4200	NaN
1	125/80	75	10000	NaN
2	125/80	75	10000	NaN
3	140/90	85	3000	Sleep Apnea
4	140/90	85	3000	Sleep Apnea

```
[6]: df.tail()
```

```
[6]:
```

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	\
369	370	Female	59	Nurse	8.1	9	
370	371	Female	59	Nurse	8.0	9	
371	372	Female	59	Nurse	8.1	9	
372	373	Female	59	Nurse	8.1	9	
373	374	Female	59	Nurse	8.1	9	

	Physical Activity Level	Stress Level	BMI Category	Blood Pressure	\
369	75	3	Overweight	140/95	
370	75	3	Overweight	140/95	
371	75	3	Overweight	140/95	
372	75	3	Overweight	140/95	
373	75	3	Overweight	140/95	

	Heart Rate	Daily Steps	Sleep Disorder
369	68	7000	Sleep Apnea
370	68	7000	Sleep Apnea
371	68	7000	Sleep Apnea
372	68	7000	Sleep Apnea
373	68	7000	Sleep Apnea

```
[7]: df.head(10)
```


[7]:

	Person ID	Gender	Age	Occupation	Sleep Duration	\
0	1	Male	27	Software Engineer	6.1	
1	2	Male	28	Doctor	6.2	
2	3	Male	28	Doctor	6.2	
3	4	Male	28	Sales Representative	5.9	
4	5	Male	28	Sales Representative	5.9	
5	6	Male	28	Software Engineer	5.9	
6	7	Male	29	Teacher	6.3	
7	8	Male	29	Doctor	7.8	
8	9	Male	29	Doctor	7.8	
9	10	Male	29	Doctor	7.8	

	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category	\
0	6	42	6	Overweight	
1	6	60	8	Normal	
2	6	60	8	Normal	
3	4	30	8	Obese	
4	4	30	8	Obese	
5	4	30	8	Obese	
6	6	40	7	Obese	
7	7	75	6	Normal	
8	7	75	6	Normal	
9	7	75	6	Normal	

	Blood Pressure	Heart Rate	Daily Steps	Sleep Disorder
0	126/83	77	4200	NaN
1	125/80	75	10000	NaN
2	125/80	75	10000	NaN
3	140/90	85	3000	Sleep Apnea
4	140/90	85	3000	Sleep Apnea
5	140/90	85	3000	Insomnia
6	140/90	82	3500	Insomnia
7	120/80	70	8000	NaN
8	120/80	70	8000	NaN
9	120/80	70	8000	NaN

[8]: df.tail(10)

```
[8]:
```

	Person ID	Gender	Age	Occupation	Sleep Duration	Quality of Sleep	\
364	365	Female	59	Nurse	8.0	9	
365	366	Female	59	Nurse	8.0	9	
366	367	Female	59	Nurse	8.1	9	
367	368	Female	59	Nurse	8.0	9	
368	369	Female	59	Nurse	8.1	9	
369	370	Female	59	Nurse	8.1	9	
370	371	Female	59	Nurse	8.0	9	
371	372	Female	59	Nurse	8.1	9	
372	373	Female	59	Nurse	8.1	9	
373	374	Female	59	Nurse	8.1	9	

	Physical Activity Level	Stress Level	BMI Category	Blood Pressure	\
364		75	3	Overweight	140/95
365		75	3	Overweight	140/95
366		75	3	Overweight	140/95
367		75	3	Overweight	140/95
368		75	3	Overweight	140/95
369		75	3	Overweight	140/95
370		75	3	Overweight	140/95
371		75	3	Overweight	140/95
372		75	3	Overweight	140/95
373		75	3	Overweight	140/95

	Heart Rate	Daily Steps	Sleep Disorder
364	68	7000	Sleep Apnea
365	68	7000	Sleep Apnea
366	68	7000	Sleep Apnea
367	68	7000	Sleep Apnea
368	68	7000	Sleep Apnea
369	68	7000	Sleep Apnea
370	68	7000	Sleep Apnea
371	68	7000	Sleep Apnea
372	68	7000	Sleep Apnea
373	68	7000	Sleep Apnea

```
[9]: df.info()
```

```
[9]: <class 'pandas.core.frame.DataFrame'>
RangeIndex: 374 entries, 0 to 373
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Person ID                            374 non-null    int64
1   Gender                               374 non-null    object
2   Age                                   374 non-null    int64
3   Occupation                           374 non-null    object
4   Sleep Duration                       374 non-null    float64
5   Quality of Sleep                     374 non-null    int64
6   Physical Activity Level              374 non-null    int64
7   Stress Level                         374 non-null    int64
8   BMI Category                         374 non-null    object
9   Blood Pressure                       374 non-null    object
10  Heart Rate                           374 non-null    int64
11  Daily Steps                          374 non-null    int64
12  Sleep Disorder                       155 non-null    object
dtypes: float64(1), int64(7), object(5)
memory usage: 38.1+ KB
```

```
[10]: df.columns
```

```
[10]: Index(['Person ID', 'Gender', 'Age', 'Occupation', 'Sleep Duration',  
         'Quality of Sleep', 'Physical Activity Level', 'Stress Level',  
         'BMI Category', 'Blood Pressure', 'Heart Rate', 'Daily Steps',  
         'Sleep Disorder'],  
        dtype='object')
```

```
[11]: # to access only one particular column  
df[['Sleep Duration']]
```

```
[11]:      Sleep Duration  
0          6.1  
1          6.2  
2          6.2  
3          5.9  
4          5.9  
..         ...  
369        8.1  
370        8.0  
371        8.1  
372        8.1  
373        8.1  
  
[374 rows x 1 columns]
```

```
[12]: # to access only one particular column  
df[['Quality of Sleep']]
```

```
[12]:      Quality of Sleep  
0          6  
1          6  
2          6  
3          4  
4          4  
..         ...  
369        9  
370        9  
371        9  
372        9  
373        9  
  
[374 rows x 1 columns]
```

```
[13]: # to access only one particular column  
df[['Stress Level']]
```

```
[13]:      Stress Level
      0          6
      1          8
      2          8
      3          8
      4          8
      ..      ...
    369          3
    370          3
    371          3
    372          3
    373          3

[374 rows x 1 columns]
```

```
[14]: # to access only one particular column
      df[['Sleep Disorder']]
```

```
[14]:      Sleep Disorder
      0          NaN
      1          NaN
      2          NaN
      3      Sleep Apnea
      4      Sleep Apnea
      ..      ...
    369      Sleep Apnea
    370      Sleep Apnea
    371      Sleep Apnea
    372      Sleep Apnea
    373      Sleep Apnea

[374 rows x 1 columns]
```

```
[15]: df.duplicated().sum()
```

```
[15]: np.int64(0)
```

```
[16]: df[['Sleep Duration']][4:15]
```

```
[16]:      Sleep Duration
      4          5.9
      5          5.9
      6          6.3
      7          7.8
      8          7.8
      9          7.8
     10          6.1
     11          7.8
     12          6.1
     13          6.0
     14          6.0
```

```
[17]: df.dtypes
```

```
[17]: Person ID          int64
      Gender           object
      Age             int64
      Occupation       object
      Sleep Duration   float64
      Quality of Sleep  int64
      Physical Activity Level int64
      Stress Level      int64
      BMI Category     object
      Blood Pressure    object
      Heart Rate        int64
      Daily Steps       int64
      Sleep Disorder    object
      dtype: object
```

```
[18]: df.describe()
```

```
[18]:
```

	Person ID	Age	Sleep Duration	Quality of Sleep \
count	374.000000	374.000000	374.000000	374.000000
mean	187.500000	42.184492	7.132086	7.312834
std	108.108742	8.673133	0.795657	1.196956
min	1.000000	27.000000	5.800000	4.000000
25%	94.250000	35.250000	6.400000	6.000000
50%	187.500000	43.000000	7.200000	7.000000
75%	280.750000	50.000000	7.800000	8.000000
max	374.000000	59.000000	8.500000	9.000000

	Physical Activity Level	Stress Level	Heart Rate	Daily Steps
count	374.000000	374.000000	374.000000	374.000000
mean	59.171123	5.385027	70.165775	6816.844920
std	20.830804	1.774526	4.135676	1617.915679
min	30.000000	3.000000	65.000000	3000.000000
25%	45.000000	4.000000	68.000000	5600.000000
50%	60.000000	5.000000	70.000000	7000.000000
75%	75.000000	7.000000	72.000000	8000.000000
max	90.000000	8.000000	86.000000	10000.000000

```
[19]: # Fill missing Sleep Disorder values
      df['Sleep Disorder'] = df['Sleep Disorder'].fillna('No Disorder')
      df['Sleep Disorder'].value_counts()
```

```
[19]: Sleep Disorder
      No Disorder    219
      Sleep Apnea    78
      Insomnia       77
      Name: count, dtype: int64
```

```
[20]: df.dropna()
```

```
[20]:
```

	Person ID	Gender	Age	Occupation	Sleep Duration \
0	1	Male	27	Software Engineer	6.1
1	2	Male	28	Doctor	6.2
2	3	Male	28	Doctor	6.2
3	4	Male	28	Sales Representative	5.9
4	5	Male	28	Sales Representative	5.9
..	...	...	...	...	...
369	370	Female	59	Nurse	8.1
370	371	Female	59	Nurse	8.0
371	372	Female	59	Nurse	8.1
372	373	Female	59	Nurse	8.1
373	374	Female	59	Nurse	8.1

	Quality of Sleep	Physical Activity Level	Stress Level	BMI Category \
0	6	42	6	Overweight
1	6	60	8	Normal
2	6	60	8	Normal
3	4	30	8	Obese
4	4	30	8	Obese
..	...	...	...	...
369	9	75	3	Overweight
370	9	75	3	Overweight
371	9	75	3	Overweight
372	9	75	3	Overweight
373	9	75	3	Overweight

	Blood Pressure	Heart Rate	Daily Steps	Sleep Disorder
0	126/83	77	4200	No Disorder
1	125/80	75	10000	No Disorder
2	125/80	75	10000	No Disorder
3	140/90	85	3000	Sleep Apnea
4	140/90	85	3000	Sleep Apnea
..	...	...	...	...
369	140/95	68	7000	Sleep Apnea
370	140/95	68	7000	Sleep Apnea
371	140/95	68	7000	Sleep Apnea
372	140/95	68	7000	Sleep Apnea
373	140/95	68	7000	Sleep Apnea

[374 rows x 13 columns]

Section 1 — Descriptive Statistics

Task 1: Measures of Central Tendency

```
[21]: print("Sleep Duration:")
print("Mean:", df['Sleep Duration'].mean())
print("Median:", df['Sleep Duration'].median())
print("Mode:", df['Sleep Duration'].mode()[0])
```

```
[21]: Sleep Duration:
Mean: 7.132085561497325
Median: 7.2
Mode: 7.2
```

```
[22]: print("\nQuality of Sleep:")
      print("Mean:", df['Quality of Sleep'].mean())
      print("Median:", df['Quality of Sleep'].median())
      print("Mode:", df['Quality of Sleep'].mode()[0])
```

```
[22]:
      Quality of Sleep:
      Mean: 7.31283422459893
      Median: 7.0
      Mode: 8
```

```
[23]: print("\nStress Level:")
      print("Mean:", df['Stress Level'].mean())
      print("Median:", df['Stress Level'].median())
      print("Mode:", df['Stress Level'].mode()[0])
```

```
[23]:
      Stress Level:
      Mean: 5.385026737967914
      Median: 5.0
      Mode: 3
```

```
[24]: # Central Tendency Summary Table
      numeric_cols = ['Sleep Duration', 'Quality of Sleep', 'Physical Activity Level',
                      'Stress Level', 'Heart Rate', 'Daily Steps']

      result = pd.DataFrame({
          'Variable': numeric_cols,
          'Mean': [round(df[c].mean(), 3) for c in numeric_cols],
          'Median': [round(df[c].median(), 3) for c in numeric_cols],
          'Mode': [round(df[c].mode()[0], 3) for c in numeric_cols]
      })

      print(result.to_string(index=False))
```

```
[24]:
```

	Variable	Mean	Median	Mode
	Sleep Duration	7.132	7.2	7.2
	Quality of Sleep	7.313	7.0	8.0
	Physical Activity Level	59.171	60.0	60.0
	Stress Level	5.385	5.0	3.0
	Heart Rate	70.166	70.0	68.0
	Daily Steps	6816.845	7000.0	8000.0

Task 2: Measures of Dispersion

```
[25]: df2 = pd.read_csv('Sleep_health_and_lifestyle_dataset.csv')
numeric_df = df2[numeric_cols]

range_val = numeric_df.max() - numeric_df.min()
std_dev   = numeric_df.std()
iqr       = numeric_df.quantile(0.75) - numeric_df.quantile(0.25)
cv        = (std_dev / numeric_df.mean()) * 100

result = pd.DataFrame({
    'Variable': numeric_df.columns,
    'Range':    range_val.values,
    'Std Dev':  std_dev.values,
    'IQR':      iqr.values,
    'CV (%)':   cv.values
})

result = result.round(3)
print(result.to_string(index=False))
```

```
[25]:
```

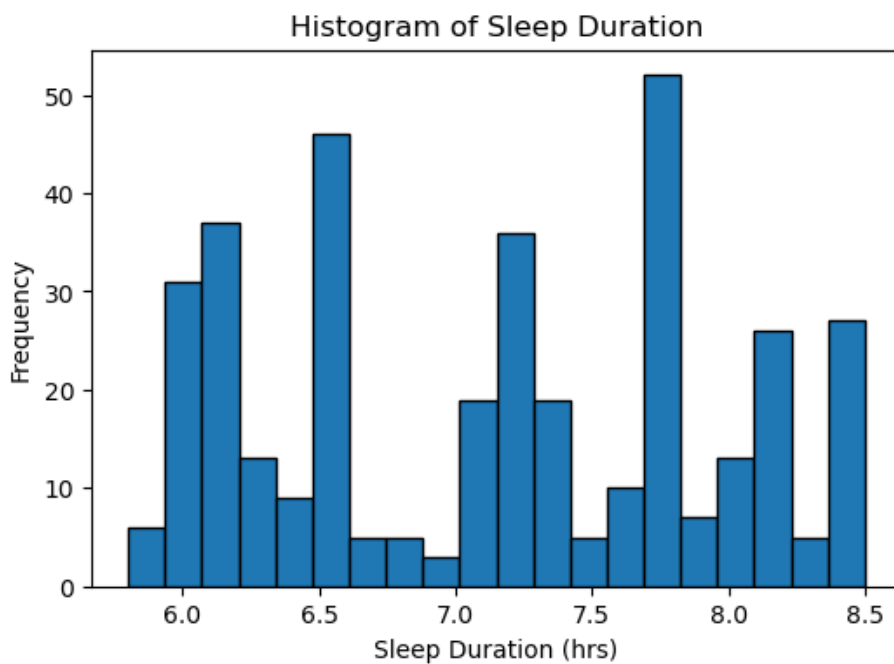
	Variable	Range	Std Dev	IQR	CV (%)
	Sleep Duration	2.7	0.796	1.4	11.156
	Quality of Sleep	5.0	1.197	2.0	16.368
	Physical Activity Level	60.0	20.831	30.0	35.204
	Stress Level	5.0	1.775	3.0	32.953
	Heart Rate	21.0	4.136	4.0	5.894
	Daily Steps	7000.0	1617.916	2400.0	23.734

Section 2 — Data Visualisation

Task 3: Histograms with KDE

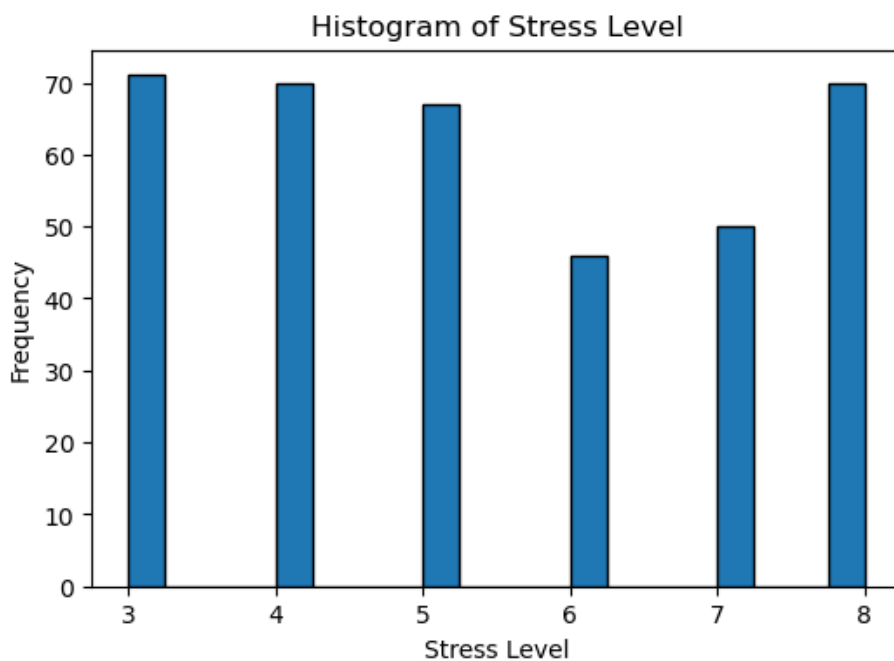
```
[26]: plt.figure(figsize=(6, 4))
plt.hist(df['Sleep Duration'], bins=20, edgecolor='black')
plt.title("Histogram of Sleep Duration")
plt.xlabel("Sleep Duration (hrs)")
plt.ylabel("Frequency")
plt.show()
```

```
[26]: <matplotlib figure>
```

```
[27]: plt.figure(figsize=(6, 4))
plt.hist(df['Stress Level'], bins=20, edgecolor='black')
plt.title("Histogram of Stress Level")
plt.xlabel("Stress Level")
plt.ylabel("Frequency")
plt.show()
```

```
[27]: <matplotlib figure>
```



```
[28]: plt.figure(figsize=(6, 4))
plt.hist(df['Quality of Sleep'], bins=20, edgecolor='black')
plt.title("Histogram of Quality of Sleep")
plt.xlabel("Quality of Sleep")
plt.ylabel("Frequency")
plt.show()
```

[28]: <matplotlib figure>



```
[29]: fig, axes = plt.subplots(1, 3, figsize=(14, 4))

cols_hist = ['Sleep Duration', 'Stress Level', 'Quality of Sleep']

# Different colors for each plot
colors = ['#4C72B0', '#E69F00', '#2CA25F'] # blue, yellow/orange, green

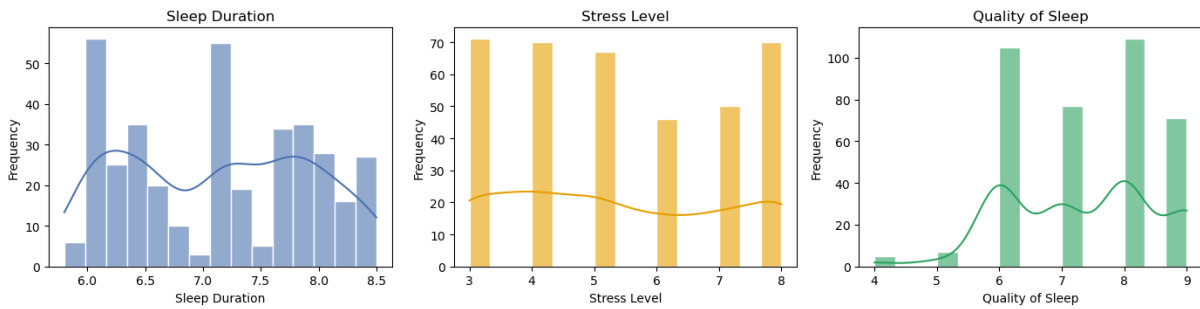
for ax, col, color in zip(axes, cols_hist, colors):
    sns.histplot(
        data=df,
        x=col,
        bins=15,
        kde=True,
        ax=ax,
        color=color,
        edgecolor='white', # gives sharp bars like your image
        alpha=0.6         # slight transparency like your image
    )
    ax.set_title(col)
    ax.set_xlabel(col)
    ax.set_ylabel('Frequency')

plt.suptitle('Histograms with Kernel Density Estimation',
             fontsize=13, fontweight='bold')

plt.tight_layout()
plt.show()
```

[29]: <matplotlib figure>

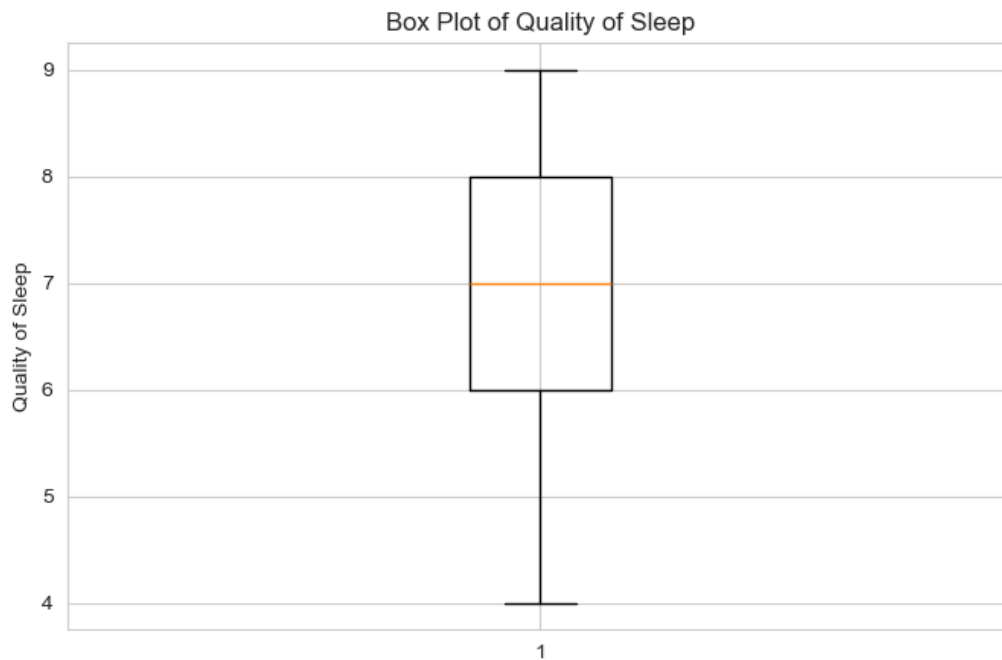
Histograms with Kernel Density Estimation



Task 4: Box Plots

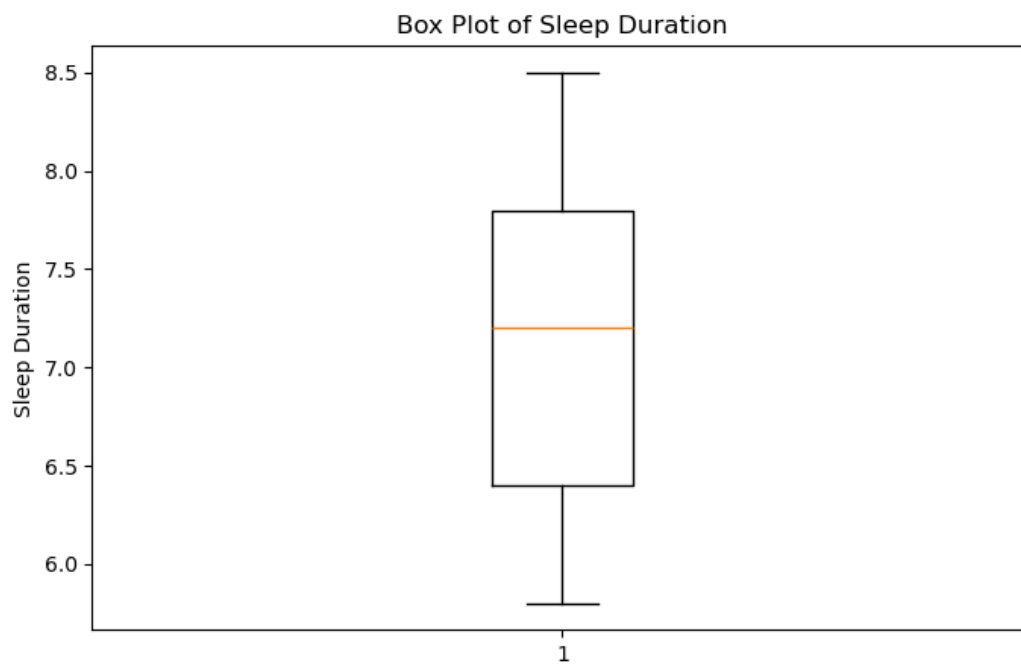
```
[30]: plt.figure(figsize=(8, 5))
plt.boxplot(df['Quality of Sleep'], vert=True)
plt.title("Box Plot of Quality of Sleep")
plt.ylabel("Quality of Sleep")
plt.show()
```

[30]: <matplotlib figure>



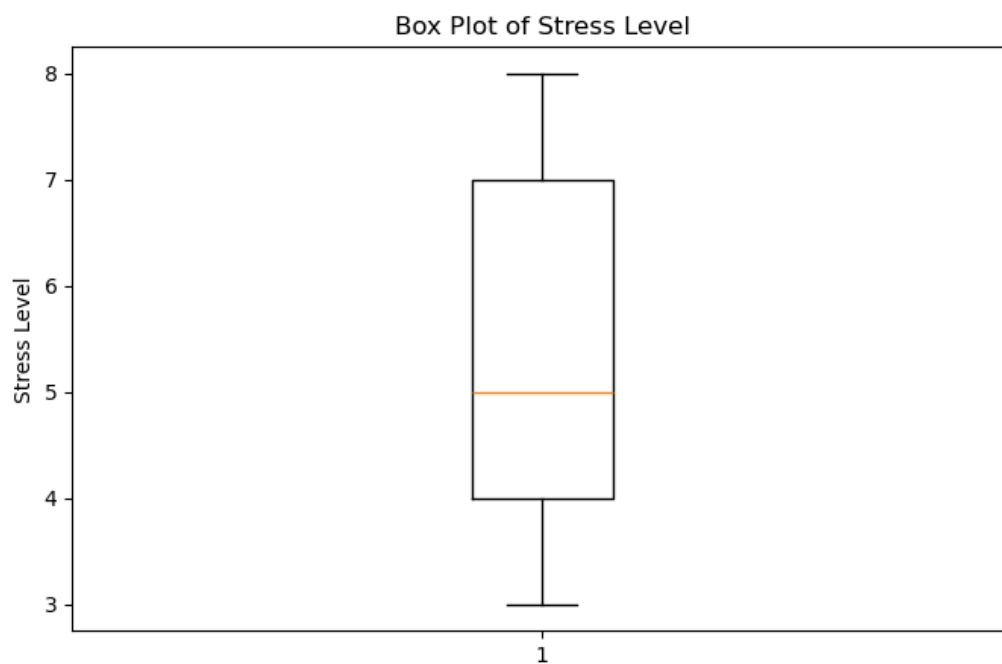
```
[31]: plt.figure(figsize=(8, 5))
plt.boxplot(df['Sleep Duration'], vert=True)
plt.title("Box Plot of Sleep Duration")
plt.ylabel("Sleep Duration")
plt.show()
```

[31]: <matplotlib figure>



```
[32]: plt.figure(figsize=(8, 5))
plt.boxplot(df['Stress Level'], vert=True)
plt.title("Box Plot of Stress Level")
plt.ylabel("Stress Level")
plt.show()
```

[32]: <matplotlib figure>



```
[33]: # Box Plots – Overall, by Gender, by Sleep Disorder
fig, axes = plt.subplots(1, 3, figsize=(14, 5))

# Overall
axes[0].boxplot(df['Quality of Sleep'], patch_artist=True)
axes[0].set_title('Overall Distribution')
axes[0].set_ylabel('Quality of Sleep')

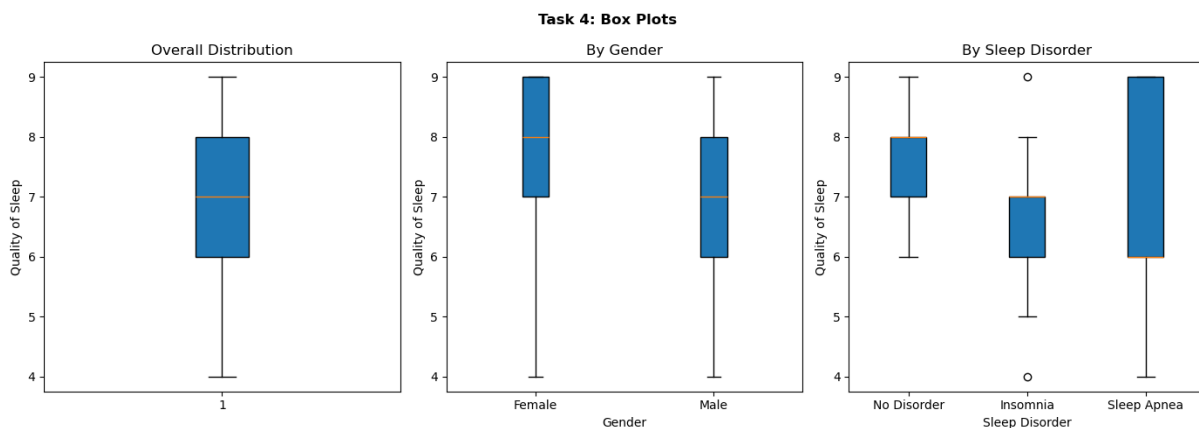
# By Gender
male = df[df['Gender'] == 'Male']['Quality of Sleep']
female = df[df['Gender'] == 'Female']['Quality of Sleep']
axes[1].boxplot([female, male], labels=['Female', 'Male'], patch_artist=True)
axes[1].set_title('By Gender')
axes[1].set_xlabel('Gender')
axes[1].set_ylabel('Quality of Sleep')

# By Sleep Disorder
groups = ['No Disorder', 'Insomnia', 'Sleep Apnea']
data_g = [df[df['Sleep Disorder'] == g]['Quality of Sleep'] for g in groups]
axes[2].boxplot(data_g, labels=['No Disorder', 'Insomnia', 'Sleep Apnea'], patch_artist=True)
axes[2].set_title('By Sleep Disorder')
axes[2].set_xlabel('Sleep Disorder')
axes[2].set_ylabel('Quality of Sleep')

plt.suptitle('Task 4: Box Plots', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.show()
```

```
[33]: C:\Users\anush\AppData\Local\Temp\ipykernel_19136\3574419662.py:12: MatplotlibDeprecationWarning: The '
axes[1].boxplot([female, male], labels=['Female', 'Male'], patch_artist=True)
C:\Users\anush\AppData\Local\Temp\ipykernel_19136\3574419662.py:20: MatplotlibDeprecationWarning: The '
axes[2].boxplot(data_g, labels=['No Disorder', 'Insomnia', 'Sleep Apnea'], patch_artist=True)
```

```
[33]: <matplotlib figure>
```

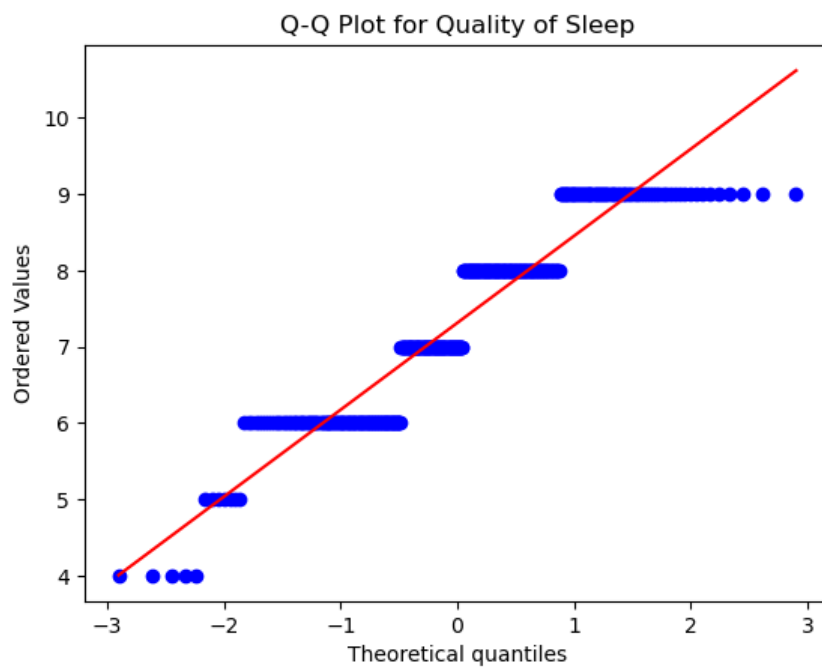


Task 5: QQ-Norm Plots

```
[34]: import scipy.stats as stats

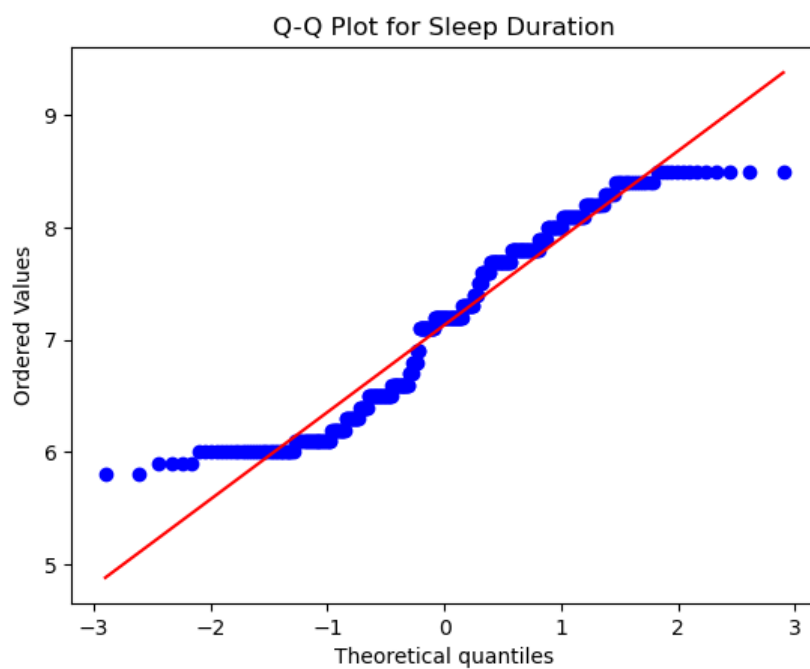
stats.probplot(df['Quality of Sleep'], dist="norm", plot=plt)
plt.title("Q-Q Plot for Quality of Sleep")
plt.show()
```

```
[34]: <matplotlib figure>
```



```
[35]: stats.probplot(df['Sleep Duration'], dist="norm", plot=plt)
      plt.title("Q-Q Plot for Sleep Duration")
      plt.show()
```

```
[35]: <matplotlib figure>
```



```
[36]: # Shapiro-Wilk Normality Test
from scipy.stats import shapiro

w_qs, p_qs = shapiro(df['Quality of Sleep'])
w_sd, p_sd = shapiro(df['Sleep Duration'])

result = pd.DataFrame({
    'Variable':      ['Quality of Sleep', 'Sleep Duration'],
    'Shapiro-Wilk W': [round(w_qs, 4), round(w_sd, 4)],
    'p-value':       [round(p_qs, 4), round(p_sd, 4)],
    'Conclusion':    [
        'Approximately Normal' if p_qs > 0.05 else 'Not Normal',
        'Approximately Normal' if p_sd > 0.05 else 'Not Normal'
    ]
})

print(result.to_string(index=False))
```

```
[36]:      Variable  Shapiro-Wilk W  p-value Conclusion
Quality of Sleep      0.8939      0.0 Not Normal
Sleep Duration      0.9358      0.0 Not Normal
```

Section 3 — Probability

Task 6: Bayes' Theorem

```
[37]: # Define events
# A = Good Sleep Quality (Quality of Sleep >= 7)
# B = No Sleep Disorder

A = df['Quality of Sleep'] >= 7
B = df['Sleep Disorder'] == 'No Disorder'

# Probabilities
P_A      = A.mean()
P_B      = B.mean()
P_A_and_B = (A & B).mean()
P_A_given_B = P_A_and_B / P_B
P_B_given_A = P_A_and_B / P_A

# Create table
result = pd.DataFrame({
    'Probability': ['P(A)', 'P(B)', 'P(A∩B)', 'P(A|B)', 'P(B|A)'],
    'Value': [P_A, P_B, P_A_and_B, P_A_given_B, P_B_given_A],
    'Interpretation': [
        'Probability of good sleep quality',
        'Probability of no sleep disorder',
        'Both good quality and no disorder',
        'Given no disorder → good quality',
        'Given good quality → no disorder'
    ]
})

result['Value'] = result['Value'].round(4)
print(result.to_string(index=False))
```

```
[37]:
```

Probability	Value	Interpretation
P(A)	0.6872	Probability of good sleep quality
P(B)	0.5856	Probability of no sleep disorder
P(A∩B)	0.4786	Both good quality and no disorder
P(A B)	0.8174	Given no disorder → good quality
P(B A)	0.6965	Given good quality → no disorder

Task 7: Normal Distribution Fit

```
[38]: from scipy.stats import norm

# Select Quality of Sleep column
qs = df['Quality of Sleep']

# Parameters
mu = qs.mean()
sigma = qs.std()

# P(Quality of Sleep > 7.5)
prob = 1 - norm.cdf(7.5, mu, sigma)

# 90th percentile
percentile_90 = norm.ppf(0.9, mu, sigma)

# Display table
result = pd.DataFrame({
    'Parameter': ['Mean ( $\mu$ )', 'Std Dev ( $\sigma$ )', 'P(Score > 7.5)', '90th Percentile'],
    'Value': [
        round(mu, 4),
        round(sigma, 4),
        round(prob, 4),
        round(percentile_90, 4)
    ]
})

print(result.to_string(index=False))
```

```
[38]:
```

Parameter	Value
Mean (μ)	7.3128
Std Dev (σ)	1.1970
P(Score > 7.5)	0.4379
90th Percentile	8.8468

Section 4 — Hypothesis Testing

Task 8: One-Sample t-Test ($H_0: \mu = 7.0$)


```
[39]: # Sample data
qs = df['Quality of Sleep']

# Hypothesized mean
mu = 7.0

# One-sample t-test
t_stat, p_val = stats.ttest_1samp(qs, mu)

# Results
sample_mean = qs.mean()

print("Sample Mean:", round(sample_mean, 3))
print("t-statistic:", round(t_stat, 4))
print("p-value:", round(p_val, 4))

# Decision
alpha = 0.05
if p_val > alpha:
    print("Decision: Fail to reject H0")
else:
    print("Decision: Reject H0")
```

```
[39]: Sample Mean: 7.313
t-statistic: 5.0544
p-value: 0.0
Decision: Reject H0
```

Task 9: Two-Sample t-Test (Male vs Female)

```
[40]: # Split data by Gender
male_qs = df[df['Gender'] == 'Male']['Quality of Sleep']
female_qs = df[df['Gender'] == 'Female']['Quality of Sleep']

# Means
mean_male = male_qs.mean()
mean_female = female_qs.mean()

# Levene test (equal variance check)
levене_stat, levене_p = stats.levене(male_qs, female_qs)

# t-test
t_stat, p_val = stats.ttest_ind(male_qs, female_qs,
                                equal_var=(levене_p > 0.05))

# Output
print("Male mean:", round(mean_male, 3))
print("Female mean:", round(mean_female, 3))
print("Levene p-value:", round(levене_p, 4))
print("t-statistic:", round(t_stat, 4))
print("p-value:", round(p_val, 4))

# Decision
alpha = 0.05
if p_val > alpha:
    print("Decision: Fail to reject H0")
else:
    print("Decision: Reject H0")
```

```
[40]: Male mean: 6.968
      Female mean: 7.665
      Levene p-value: 0.0006
      t-statistic: -5.8593
      p-value: 0.0
      Decision: Reject H0
```

Section 5 — Correlation & Regression

Task 10: Correlation Analysis

```
[41]: from scipy.stats import pearsonr
      import numpy as np
      import matplotlib.pyplot as plt
      import seaborn as sns

      # Select numeric columns
      numeric_df = df[numeric_cols]

      # Correlation matrix
      corr = numeric_df.corr()

      # Mask upper triangle
      mask = np.triu(np.ones_like(corr, dtype=bool), k=1)

      # Custom color palette (THIS is the key)
      cmap = sns.diverging_palette(10, 240, s=90, l=30, as_cmap=True)

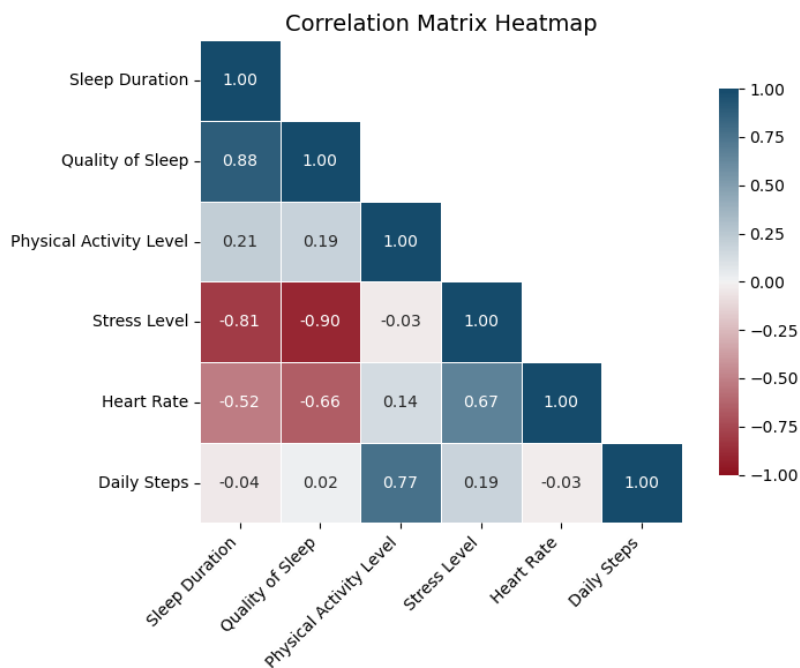
      # Plot
      plt.figure(figsize=(8, 6))

      sns.heatmap(
          corr,
          mask=mask,
          annot=True,
          cmap=cmap,
          vmin=-1,
          vmax=1,
          center=0,          # important for proper color balance
          linewidths=0.5,
          fmt=".2f",
          square=True,
          cbar_kws={"shrink": 0.8}
      )

      plt.title("Correlation Matrix Heatmap", fontsize=14)
      plt.xticks(rotation=45, ha='right')
      plt.yticks(rotation=0)

      plt.tight_layout()
      plt.show()
```

```
[41]: <matplotlib figure>
```



```
[42]: # Correlation Table – each variable vs Quality of Sleep
target = 'Quality of Sleep'

results = []
for col in numeric_cols:
    if col != target:
        r, p = pearsonr(df[col], df[target])
        sig = '✓ Significant' if p < 0.05 else '✗ Not significant'
        results.append({
            'Pair': f'{col} ↔ {target}',
            'r': round(r, 3),
            'p-value': round(p, 4),
            'Significance': sig
        })

result_df = pd.DataFrame(results)
print("\nCorrelation Table:\n")
print(result_df.to_string(index=False))
```

[42]:

Correlation Table:

	Pair	r	p-value	Significance
	Sleep Duration ↔ Quality of Sleep	0.883	0.0000	✓ Significant
	Physical Activity Level ↔ Quality of Sleep	0.193	0.0002	✓ Significant
	Stress Level ↔ Quality of Sleep	-0.899	0.0000	✓ Significant
	Heart Rate ↔ Quality of Sleep	-0.660	0.0000	✓ Significant
	Daily Steps ↔ Quality of Sleep	0.017	0.7462	✗ Not significant

Task 11: Simple Linear Regression

```
[43]: from sklearn.linear_model import LinearRegression

# Independent (X) → Stress Level
# Dependent (y) → Quality of Sleep
X = df['Stress Level']
y = df['Quality of Sleep']

# Model
model = LinearRegression()
model.fit(X, y)

# Parameters
intercept = model.intercept_
slope      = model.coef_[0]
r2         = model.score(X, y)

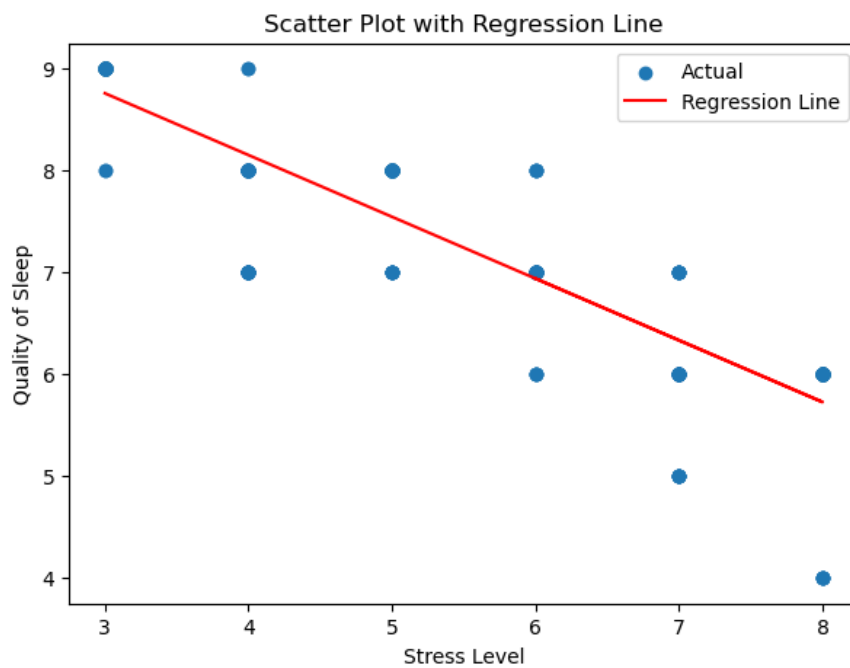
# Print results (table style)
print("--- Regression Results ---")
print("Intercept ( $\beta_0$ ):", round(intercept, 3))
print("Slope ( $\beta_1$ ):",      round(slope, 3))
print("R2:",                round(r2, 3))

# Predictions
y_pred = model.predict(X)
```

```
[43]: --- Regression Results ---
Intercept ( $\beta_0$ ): 10.577
Slope ( $\beta_1$ ): -0.606
R2: 0.808
```

```
[44]: # Scatter + Regression Line
plt.figure(figsize=(7, 5))
plt.scatter(X, y, label='Actual')
plt.plot(X, y_pred, color='red', label='Regression Line')
plt.xlabel('Stress Level')
plt.ylabel('Quality of Sleep')
plt.title('Scatter Plot with Regression Line')
plt.legend()
plt.show()
```

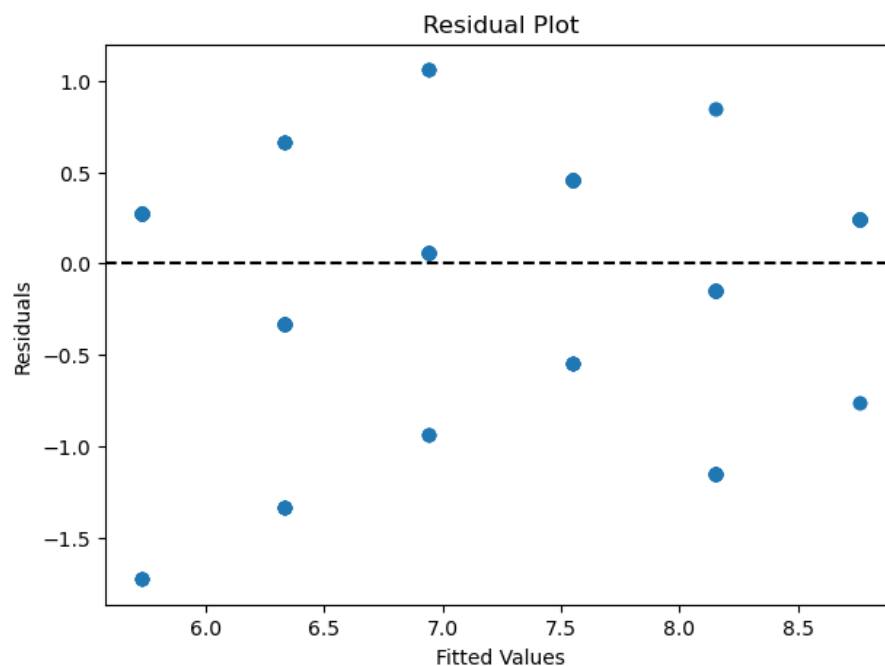
```
[44]: <matplotlib figure>
```



```
[45]: # Residual Plot
      residuals = y - y_pred

      plt.figure(figsize=(7, 5))
      plt.scatter(y_pred, residuals)
      plt.axhline(y=0, color='black', linestyle='--')
      plt.xlabel('Fitted Values')
      plt.ylabel('Residuals')
      plt.title('Residual Plot')
      plt.show()
```

[45]: <matplotlib figure>



Section 6 — Chi-Square Test for Independence

Task 12: Gender × Sleep Disorder

```
[46]: from scipy.stats import chi2_contingency

# Contingency table
table = pd.crosstab(df['Gender'], df['Sleep Disorder'])

# Chi-square test
chi2, p, dof, expected = chi2_contingency(table)

# Print contingency table
print("Contingency Table:")
print(table)
print()

# Print statistics
print("Statistic      Value")
print(f" $\chi^2$  statistic      {chi2:.4f}")
print(f"Degrees of freedom {dof}")
print(f"p-value              {p:.4f}")

# Decision
alpha = 0.05
if p < alpha:
    print("Decision ( $\alpha=0.05$ ): Reject  $H_0 \rightarrow$  Attributes are dependent")
else:
    print("Decision ( $\alpha=0.05$ ): Fail to reject  $H_0 \rightarrow$  Attributes are independent")
```

```
[46]: Contingency Table:
Sleep Disorder  Insomnia  No Disorder  Sleep Apnea
Gender
Female          36         82          67
Male            41        137          11

Statistic      Value
 $\chi^2$  statistic      54.3060
Degrees of freedom 2
p-value         0.0000
Decision ( $\alpha=0.05$ ): Reject  $H_0 \rightarrow$  Attributes are dependent
```

```
[47]: # Extension: Gender × High Quality Sleep
# High Quality = Quality of Sleep >= median

median_qs = df['Quality of Sleep'].median()
df['High_Quality'] = df['Quality of Sleep'].apply(
    lambda x: 'High Quality' if x >= median_qs else 'Low Quality'
)

# Contingency table
table2 = pd.crosstab(df['Gender'], df['High_Quality'])

# Chi-square test
chi2_2, p_2, dof_2, expected_2 = chi2_contingency(table2)

# Print
print("Contingency Table (Gender × High Quality Sleep):")
print(table2)
print()
print(f"χ² statistic      {chi2_2:.4f}")
print(f"p-value           {p_2:.4f}")

# Decision
if p_2 < alpha:
    print("Decision (α=0.05): Reject H₀ → Gender and performance are dependent")
else:
    print("Decision (α=0.05): Fail to reject H₀ → Gender and performance are independent")
```

```
[47]: Contingency Table (Gender × High Quality Sleep):
High_Quality  High Quality  Low Quality
Gender
Female                142           43
Male                  115           74

χ² statistic      10.2811
p-value           0.0013
Decision (α=0.05): Reject H₀ → Gender and performance are dependent
```